

Proiect: Lanț de Magazine Cosmetice

Student: Purenciu Diana

Grupa: 30236

1 Enunțul problemei

Dezvoltați o aplicație software care poate fi utilizată într-un lanț de magazine cu produse cosmetice. Aplicația va avea 4 tipuri de utilizatori: client, angajat, manager și administrator.

Utilizatorii de tip **client** pot efectua următoarele operații fără autentificare:

- Vizualizarea listei tuturor produselor cosmetice sortată după denumire și preț (vizualizarea include și redarea unor imagini cu produsele cosmetice);
- Căutarea unui produs cosmetic după denumire, afișându-se și lista magazinelor unde este disponibil.

Utilizatorii de tip **angajat** pot efectua următoarele operații după autentificare:

- Vizualizarea listei tuturor produselor cosmetice din magazinul unde lucrează sortată după denumire și preț (între 1 și 3 imagini pentru fiecare produs);
- Filtrarea produselor cosmetice din magazinul unde lucrează după: producător, disponibilitate, preț;
- Căutarea unui produs cosmetic după denumire:
 - În magazinul propriu, afișând toate detaliile dacă este pe stoc;
 - Dacă nu este pe stoc local, se caută în restul lanțului de magazine și se afișează unde este disponibil.
- Vânzarea unui produs cosmetic (scăderea stocului);
- Adăugarea și actualizarea stocului produselor din magazinul propriu;
- Salvarea listelor cu produsele din magazin în mai multe formate: csv, json, xml, doc.

Utilizatorii de tip **manager** pot efectua următoarele operații după autentificare:

- Toate operațiile permise clienților;
- Filtrarea produselor din tot lanțul după: magazin, producător, disponibilitate;
- Adăugarea, modificarea sau ștergerea produselor cosmetice (denumire, producător, imagini, preț achiziție și vânzare);
- Salvarea listelor cu produse în formatele: csv, json, xml, doc;
- Vizualizarea unor statistici (minim 3) legate de produse, folosind grafice.

Utilizatorii de tip **administrator** pot efectua următoarele operații după autentificare:

- Adăugarea, modificarea sau ștergerea conturilor de utilizatori;
- Vizualizarea tuturor utilizatorilor, filtrarea lor după rol și salvarea listei în format csv;
- Trimiterea de notificări automate (prin email, SMS etc.) la orice modificare a datelor de cont.

Interfața grafică va fi disponibilă în cel puțin 3 limbi internaționale.

2 Instrumente utilizate

Pentru realizarea acestui proiect, s-au utilizat următoarele tehnologii și programe:

- **Limbaj de programare:** Java 17.
- **Framework:** Spring Boot (pentru microservicii și gestionarea bazelor de date).
- **Bază de date:** MS SQL Server.
- **Instrumente de desenare:** PlantUML și DrawIO (pentru diagramele UML).
- **Testare:** Postman (pentru a testa legăturile cu serverul).
- **Mediu de dezvoltare:** IntelliJ IDEA.

3 Justificarea limbajului de programare ales

Am ales limbajul Java împreună cu framework-ul Spring Boot din următoarele motive:

- **Organizare bună (DDD):** Java folosește programarea orientată pe obiecte, ceea ce ne ajută să separăm clar regulile aplicației de codul tehnic.
- **Perfect pentru Microservicii:** Spring Boot oferă unelte integrate cu care putem crea rapid rute web (REST) prin care aplicația să comunice.
- **Lucrul ușor cu bazele de date:** Prin Spring Data JPA, clasele din cod se transformă automat în tabele în baza de date, fără să mai scriem comenzi SQL manuale.
- **Cod ordonat:** Ne ajută să împărțim codul pe straturi (Domain, Infrastructure, Services, Controllers), făcând aplicația ușor de modificat în viitor.

4 Justificarea tipului de comunicare între microservicii

Am ales să folosesc o comunicare directă (sincronă) între microservicii, folosind **REST API** (prin *RestTemplate* în Spring). Am luat această decizie deoarece:

- **Este simplu:** Trimiterea de date prin HTTP este ușor de programat și de testat.
- **Schimb standard de date:** Microserviciile comunică între ele într-un format simplu și universal recunoscut (JSON).
- **Răspuns imediat:** Pentru acțiuni critice (de exemplu, când trebuie să verificăm stocul înainte să vindem un produs), avem nevoie de un răspuns pe loc, lucru pe care comunicarea REST îl oferă eficient.

5 Descrierea diagramelor UML

5.1 Faza de analiză

5.1.1 Diagrama cazurilor de utilizare

Această diagramă arată ce poate face fiecare utilizator. Clientul are acces liber la vizualizare. Pentru Angajat, Manager și Administrator, toate acțiunile necesită autentificare prealabilă (cerință marcată cu săgeți *include*).

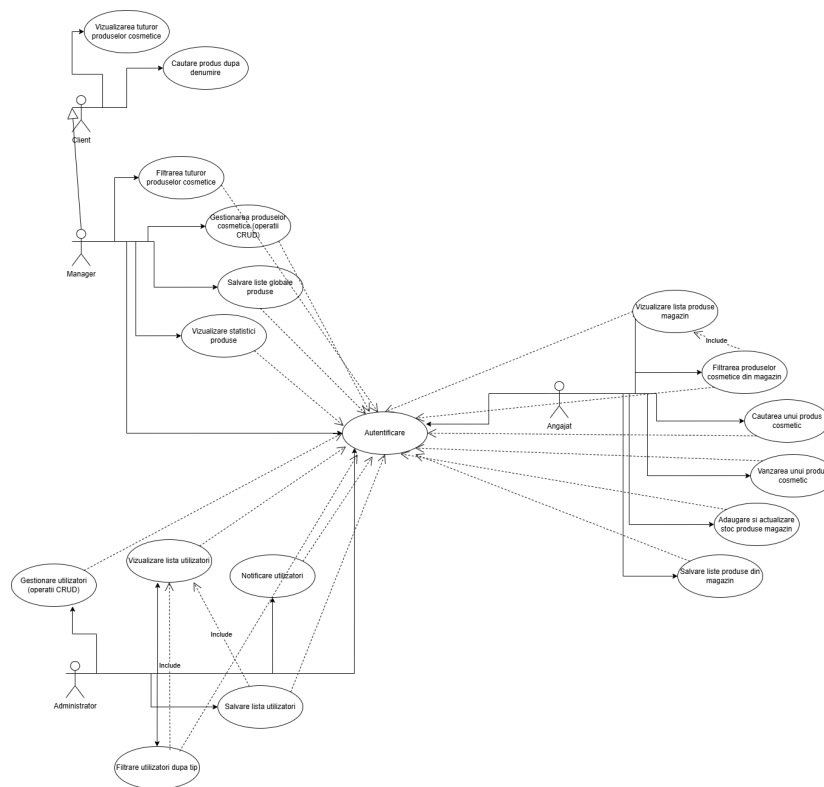


Figure 1: Diagrama cazurilor de utilizare a sistemului

5.1.2 Diagramele de activități

Aceste diagrame explică pașii exacti pentru fiecare caz de utilizare în parte. Am folosit coloane pentru a separa clar acțiunile utilizatorului de procesele rulate de sistem.

1. Autentificarea Utilizatorului

Utilizatorul introduce datele, programul le verifică în baza de date, iar apoi îl direcționează către panoul corespunzător rolului său.

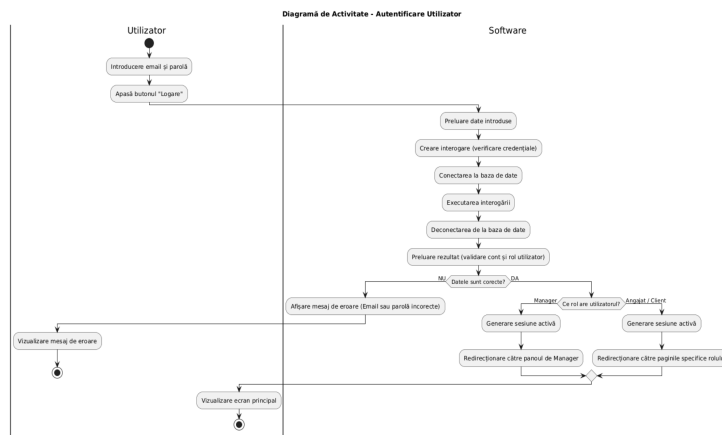


Figure 2: Diagrama de activitate: Autentificare

2. Vizualizare produse (Client)

Clientul accesează meniul, iar sistemul extrage și afișează lista de produse direct din baza de date a catalogului.

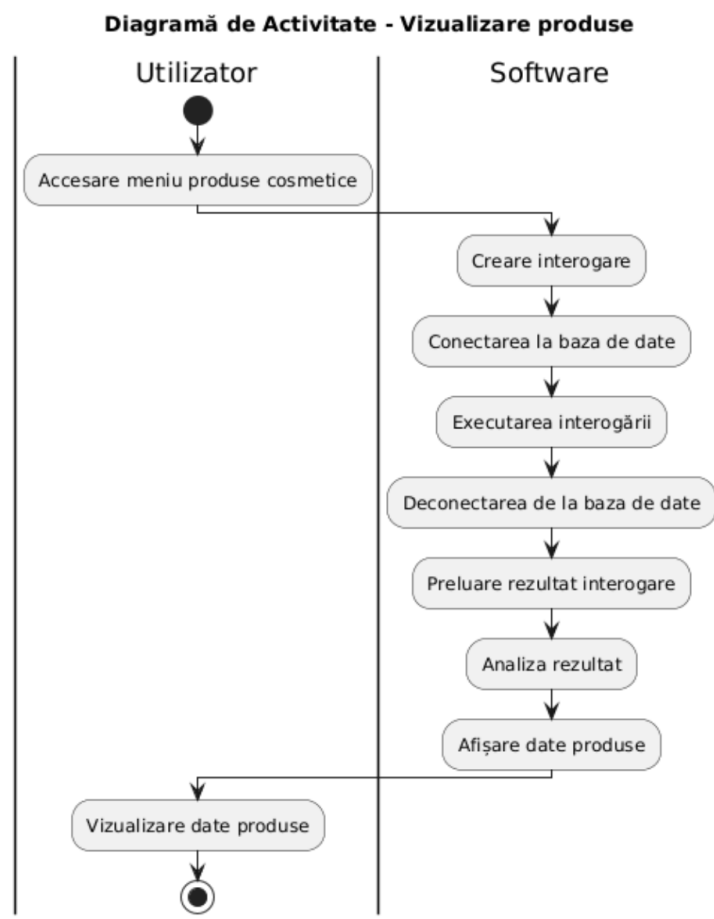


Figure 3: Diagrama de activitate: Vizualizare produse (Client)

3. Căutare produs după denumire (Client)

Clientul caută un nume, iar programul verifică disponibilitatea și afișează toate magazinele care au acel produs pe stoc.

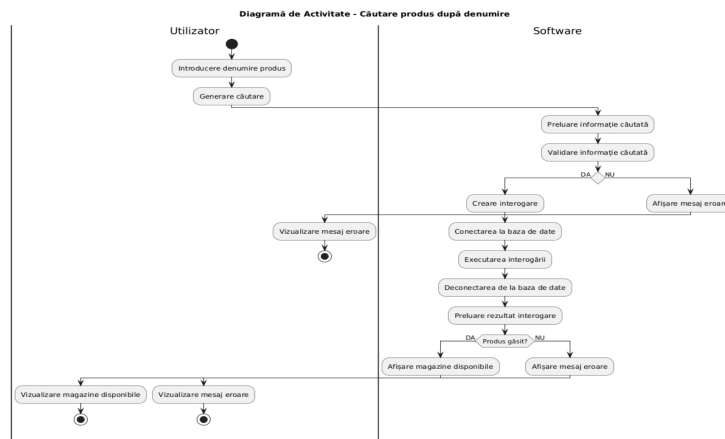


Figure 4: Diagrama de activitate: Căutare produs (Client)

4. Vizualizare produse magazin (Angajat)

Programul identifică magazinul angajatului pe baza sesiunii și aduce doar produsele disponibile în acea locație, împreună cu imaginile lor.

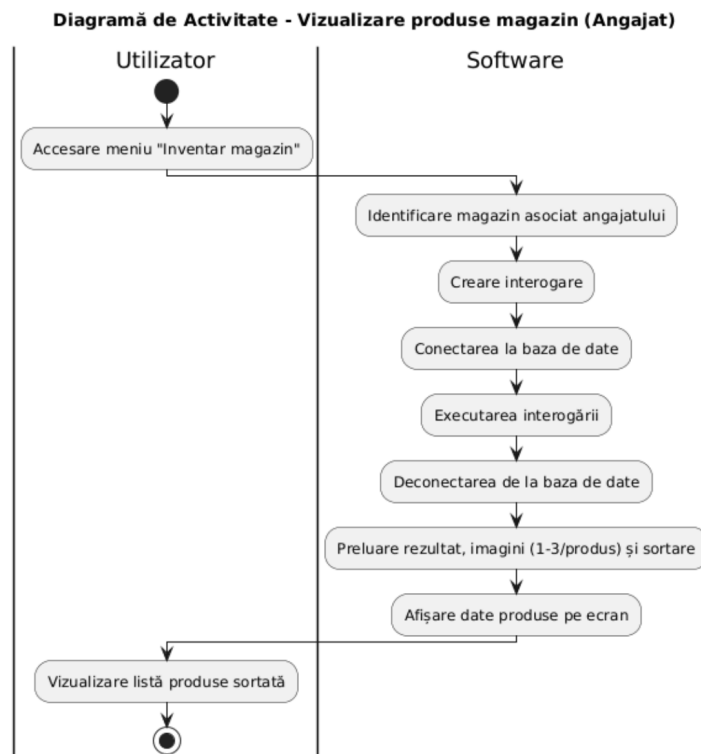


Figure 5: Diagrama de activitate: Vizualizare produse magazin (Angajat)

5. Filtrare produse magazin (Angajat)

Angajatul alege filtrele (ex. producător, preț), iar sistemul actualizează lista doar cu produsele din magazinul local care respectă cerințele.

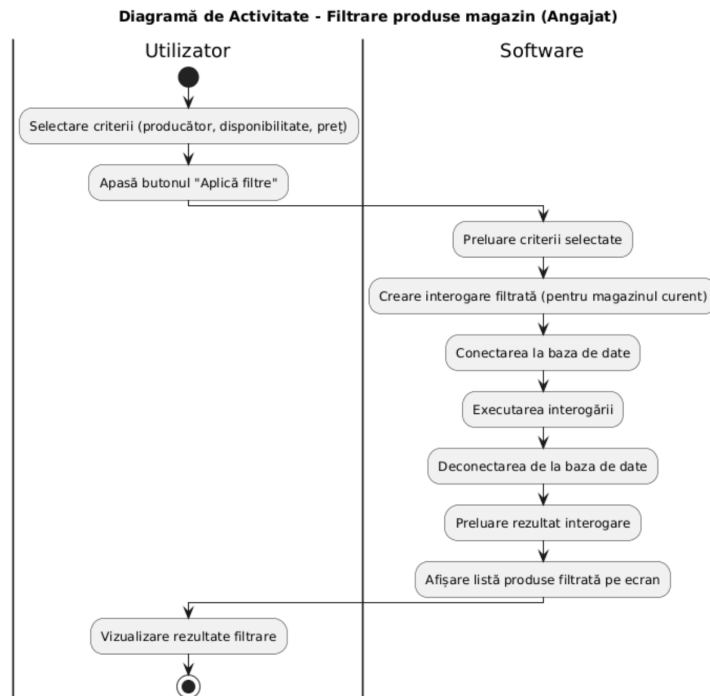


Figure 6: Diagrama de activitate: Filtrare produse magazin (Angajat)

6. Căutare avansată produs (Angajat)

Se caută produsul prioritar în magazinul angajatului. Dacă stocul este zero, programul caută automat produsul în restul magazinelor din rețea.

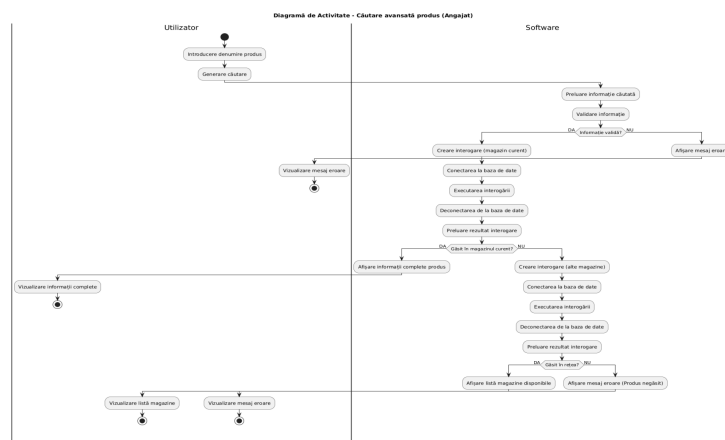


Figure 7: Diagrama de activitate: Căutare avansată (Angajat)

7. Vânzare produs cosmetic (Angajat)

Se verifică stocul curent. Dacă este disponibil, programul scade cantitatea din baza de date și confirmă efectuarea vânzării.

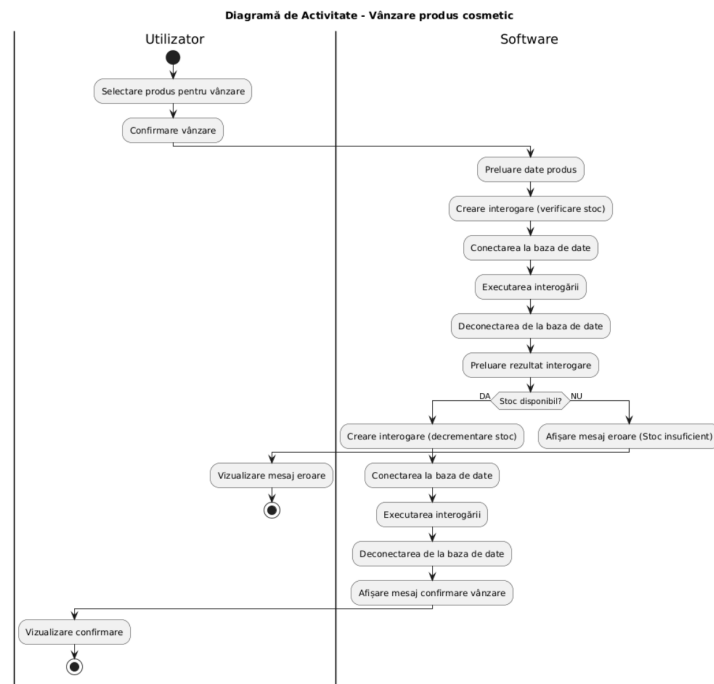


Figure 8: Diagrama de activitate: Vânzare produs (Angajat)

8. Adăugare și actualizare stoc (Angajat)

Angajatul introduce o nouă cantitate, iar programul salvează modificarea strict în stocul magazinului local.

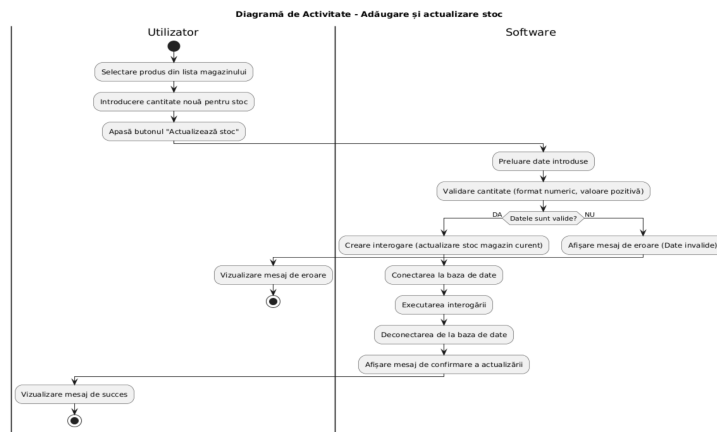


Figure 9: Diagrama de activitate: Actualizare stoc (Angajat)

9. Export situație stoc local (Angajat)

Programul colectează datele din magazinul curent și generează un fișier în formatul ales (CSV, JSON, XML sau DOC).

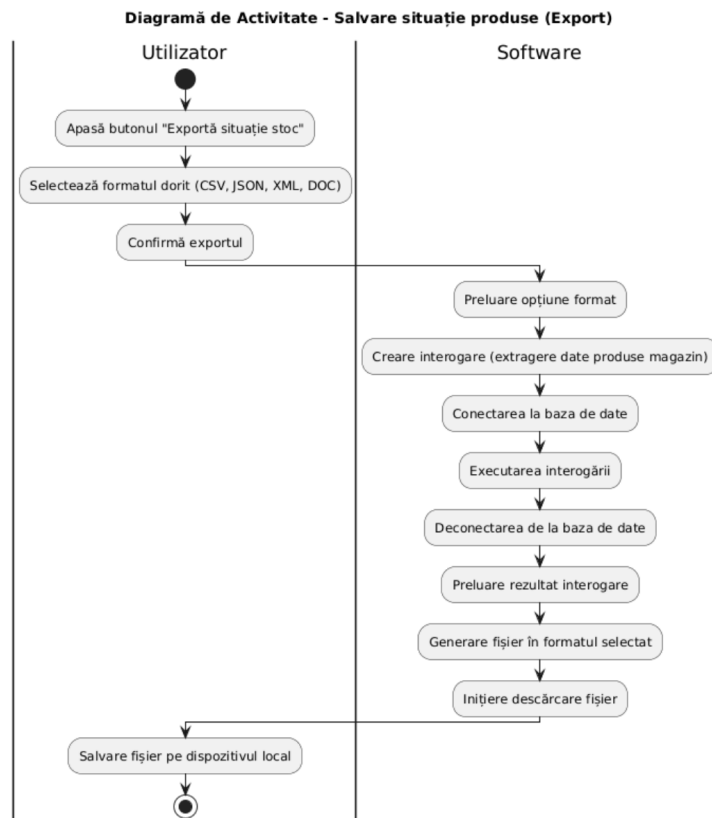


Figure 10: Diagrama de activitate: Export stoc (Angajat)

10. Operații CRUD produse (Manager)

Managerul adaugă sau modifică informațiile produselor, iar sistemul salvează aceste date în catalogul central.

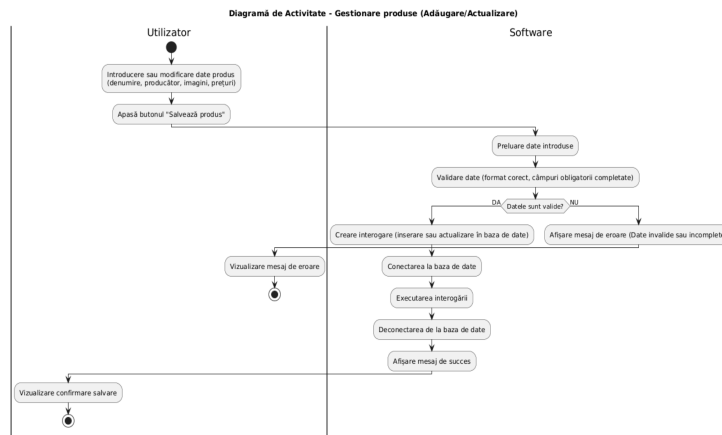


Figure 11: Diagrama de activitate: Modificare produse (Manager)

11. Filtrare globală produse (Manager)

Managerul aplică filtre la nivelul întregii rețele, putând verifica și analiza stocul din orice locație.

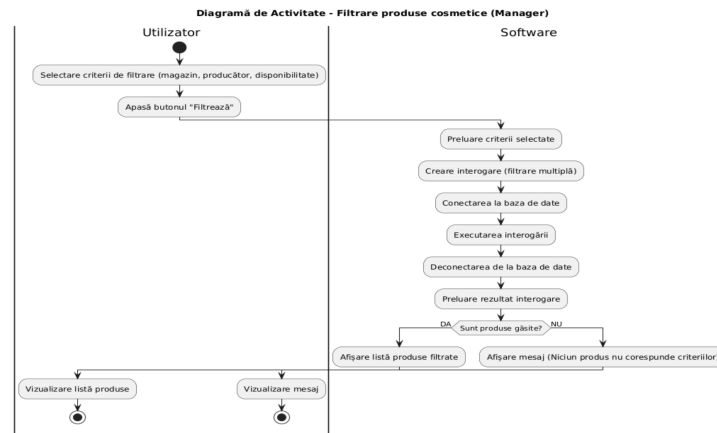


Figure 12: Diagrama de activitate: Filtrare globală (Manager)

12. Vizualizare statistici și grafice (Manager)

Programul extrage date despre vânzări și produse, efectuând calcule pentru a le afișa sub formă de grafice pe ecranul managerului.

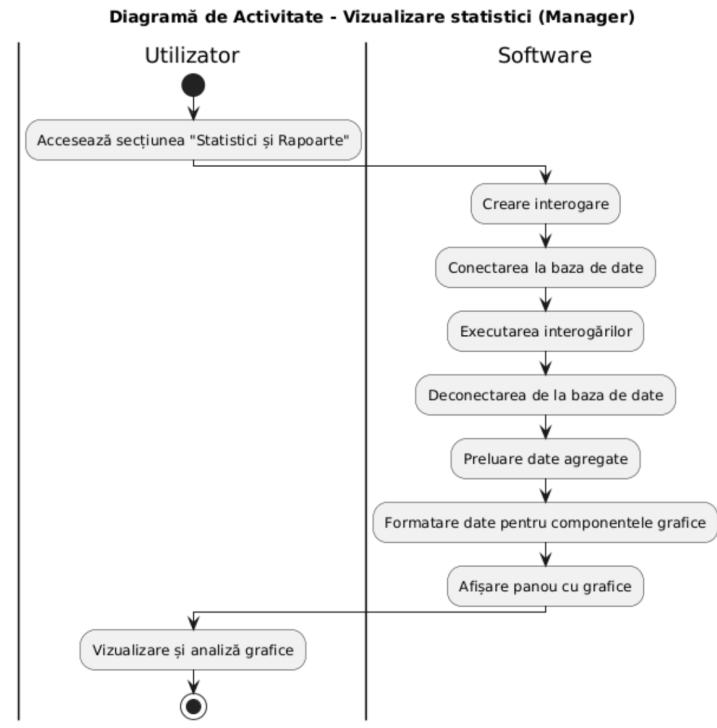


Figure 13: Diagrama de activitate: Vizualizare grafice (Manager)

13. Export situație globală (Manager)

Se generează un raport complet cu datele din toate magazinele, fiind descărcat în formatul specificat de manager.

14. Gestionare utilizatori CRUD (Administrator)

Administratorul adaugă, modifică sau șterge conturi, iar sistemul reflectă modificările în baza de date a utilizatorilor.

15. Vizualizare, filtrare și export utilizatori (Administrator)

Sunt afișate toate conturile din sistem. Lista poate fi filtrată pe roluri sau salvată într-un fișier de tip CSV.

16. Notificare modificare date autentificare (Automat)

La orice modificare a unui cont, sistemul declanșează automat un flux de trimiteri a unor mesaje de atenționare (email și SMS).

5.2 Faza de proiectare

În această etapă s-a detaliat arhitectura internă a fiecărui microserviciu, respectând principiile Domain-Driven Design (DDD).

5.2.1 Diagrame de pachete, clase și baze de date (ER) pe Microservicii

Fiecare microserviciu este organizat pe pachete (straturi) pentru decuplare:

- **Domain:** Conține regulile de business și obiectele de transfer (DTO-uri).
- **Infrastructure:** Gestionează implementarea tehnică și comunicarea cu baza de date.
- **Services:** Conține logica aplicației și orchestrează procesele.
- **Controllers:** Preia cererile REST din interfață.

Microserviciul Catalog (ProductService)

Gestionează catalogul central de produse. Baza de date salvează informațiile despre cosmetice și imaginile asociate.

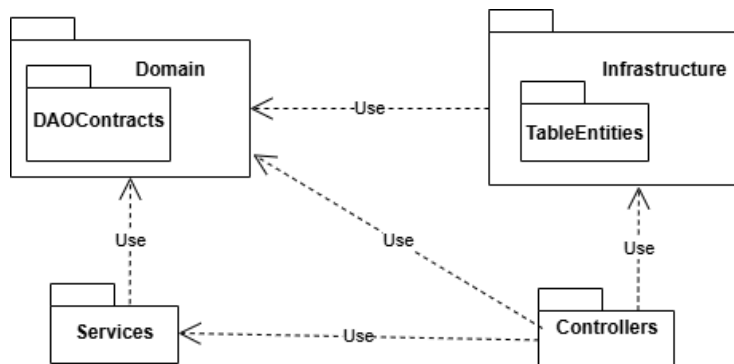


Figure 14: Diagrama de pachete: ProductService

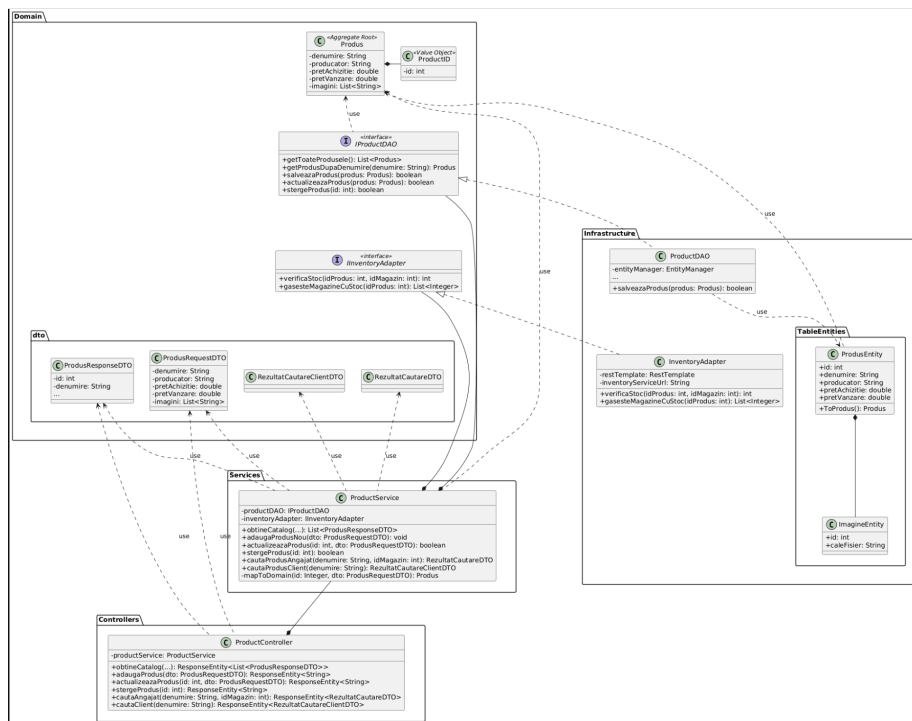


Figure 15: Diagrama de clase: ProductService

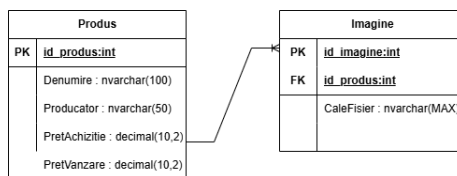


Figure 16: Diagrama bazei de date (ER): Produse și Imagini

Microserviciul de Inventar și Vânzări (InventoryService)

Se ocupă de stocurile fizice din fiecare locație. Baza de date conține legăturile dintre magazin, stoc și istoricul vânzărilor.

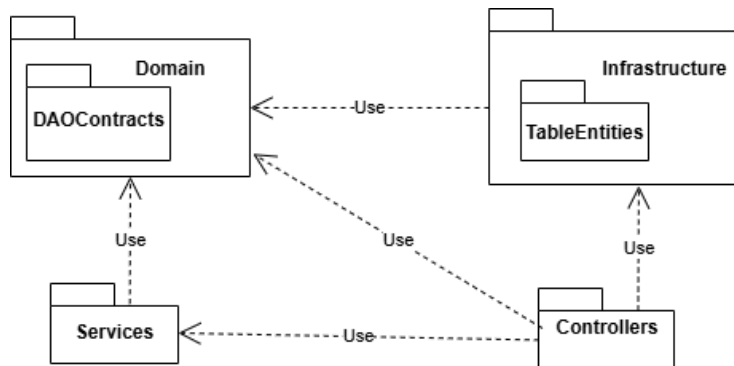


Figure 17: Diagrama de pachete: InventoryService

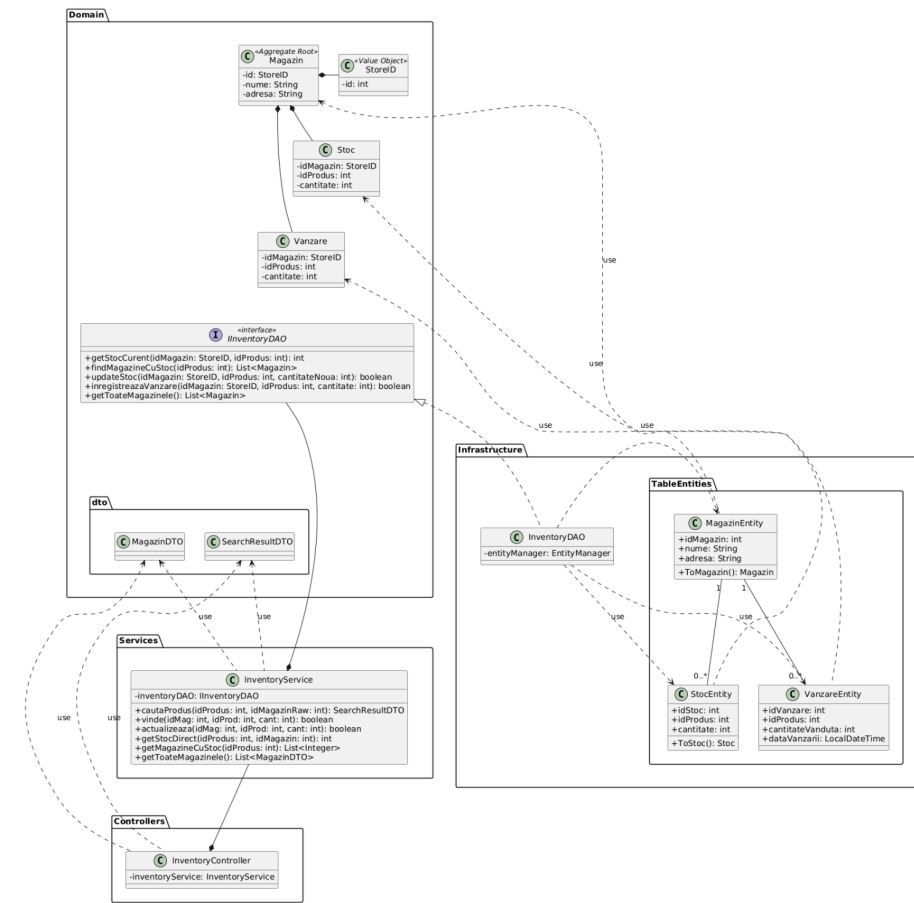


Figure 18: Diagrama de clase: InventoryService

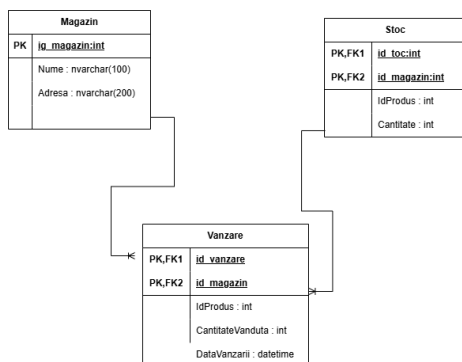


Figure 19: Diagrama bazei de date (ER): Magazin, Stoc, Vanzări

Microserviciul de Utilizatori (UserService)

Gestionează conturile și securitatea, având un model de baze de date simplificat pentru personal.

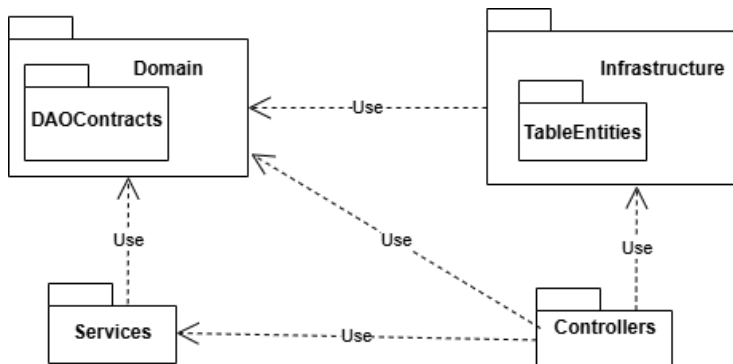


Figure 20: Diagrama de pachete: UserService

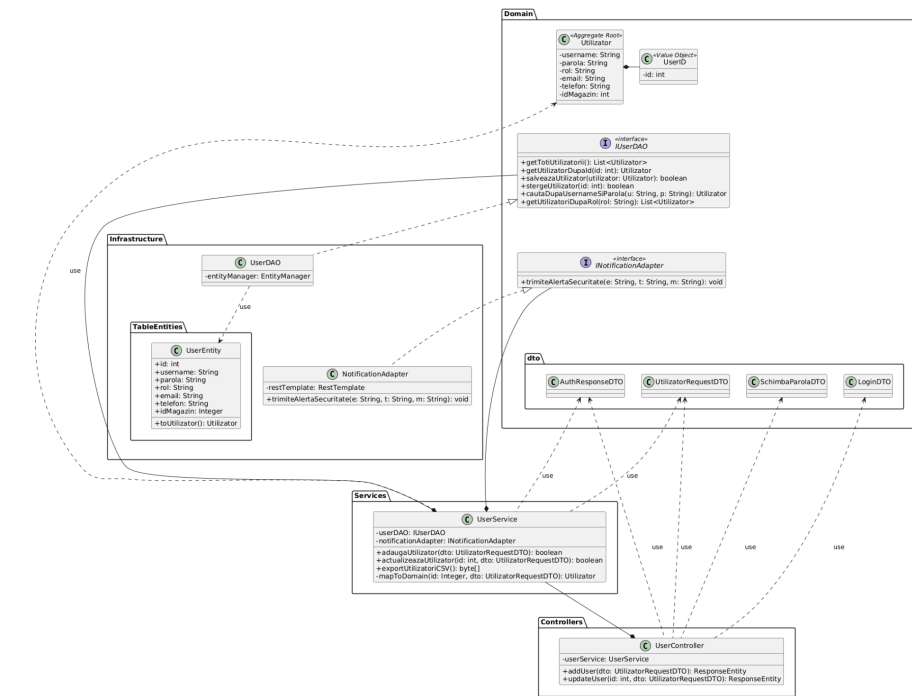


Figure 21: Diagrama de clase: UserService

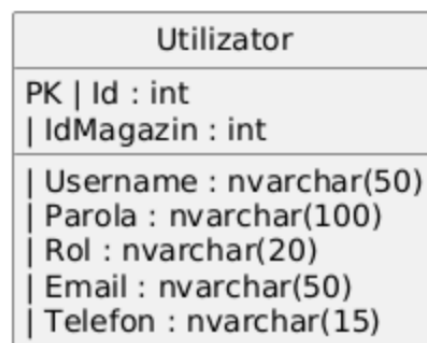


Figure 22: Diagrama bazei de date (ER): Utilizator

Microserviciul de Notificări (Notification Service)

Utilizează șablonul de proiectare **Observer**. Dispecerul notifică toți observatorii abonați (Email, SMS). Baza de date reține istoricul mesajelor.

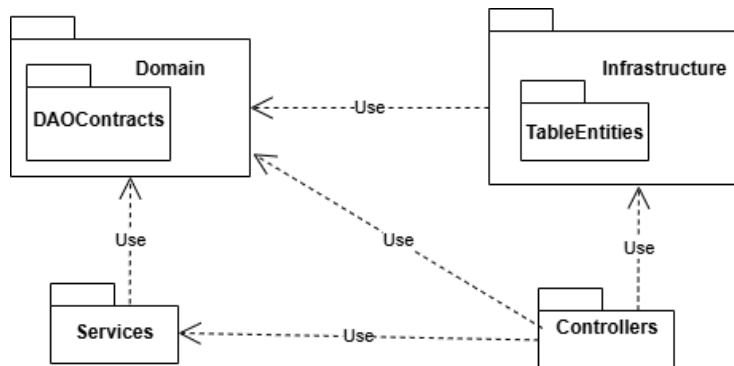


Figure 23: Diagrama de pachete: Notification Service

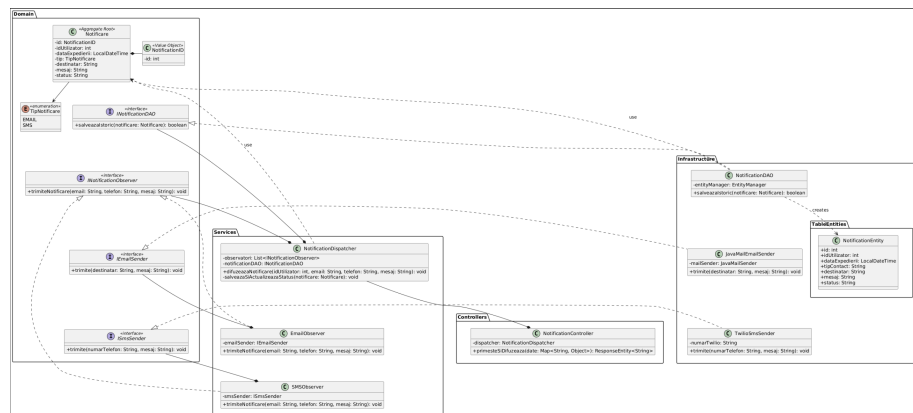


Figure 24: Diagrama de clase: Notification Service

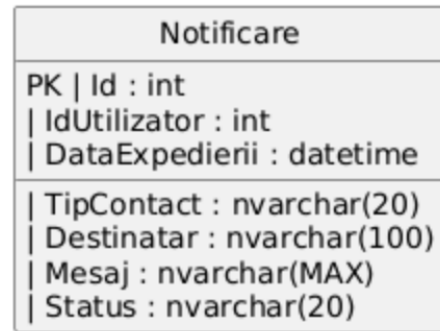


Figure 25: Diagrama bazei de date (ER): Istoric Notificări

Microserviciul de Statistici (Reporting Service)

Acest modul preia date de la alte servicii folosind un HTTP Client (Singleton) și utilizează șablonul **Factory** pentru a genera rapoarte în formatul solicitat. Nu deține o bază de date proprie.

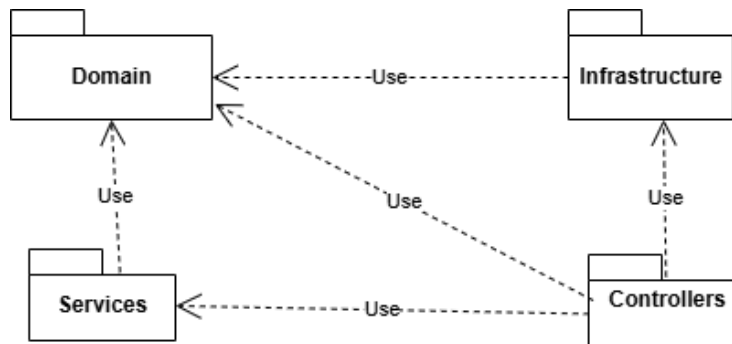


Figure 26: Diagrama de pachete: Reporting Service

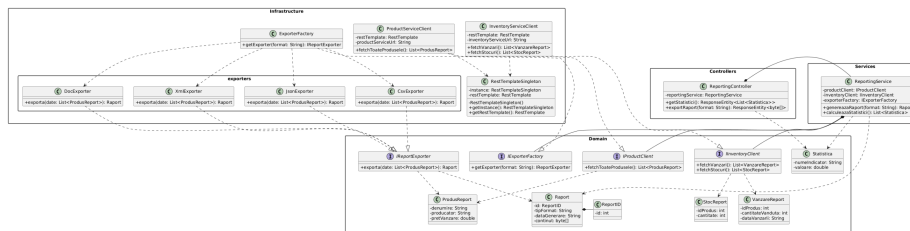


Figure 27: Diagrama de clase: Reporting Service

5.2.2 Diagrame de secvență

În această secțiune sunt prezentate diagramele de secvență, care arată modul în care componentele comunică în timp pentru a îndeplini cazurile de utilizare.

1. Vizualizare statistici (Manager)

Managerul cere datele. Serviciul de rapoarte agreghează informații din Catalog și Inventar, calculează indicatorii și trimite rezultatele sub formă de grafice.

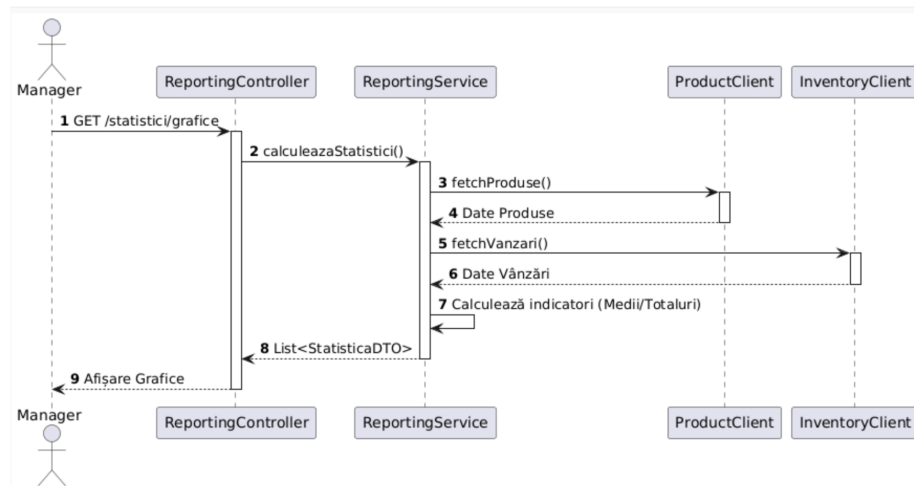


Figure 28: Diagrama de secvență: Vizualizare statistici

2. Actualizare stoc (Angajat)

Angajatul modifică cantitatea. Controllerul trimite datele către serviciu, care execută comanda de update direct în baza de date locală.

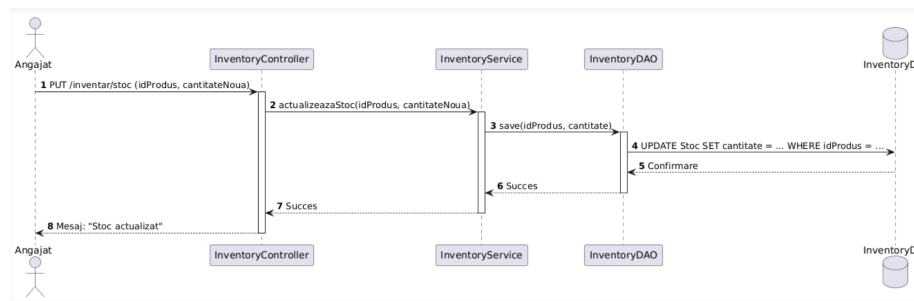


Figure 29: Diagrama de secvență: Actualizare stoc

3. Vizualizare stoc magazin (Angajat)

Sistemul preia ID-ul magazinului din cerere și interoghează DAO-ul pentru a returna lista stocurilor specifice acelei locații.

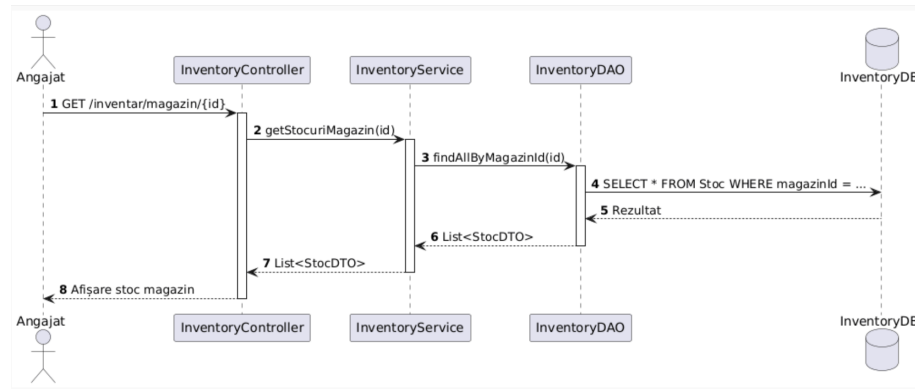


Figure 30: Diagrama de secvență: Vizualizare produse angajat

4. Căutare produs (Client)

Aplicația găsește detaliile produsului în Catalog, apoi folosește un adaptor pentru a interoga Inventarul cu privire la locațiile unde există stoc pozitiv.

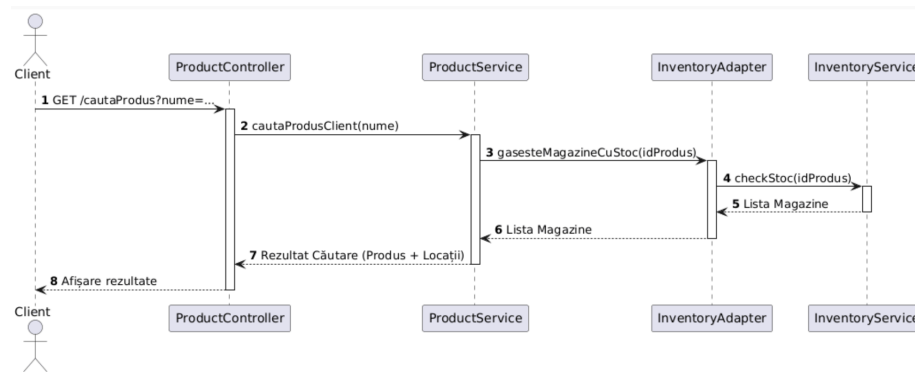


Figure 31: Diagrama de secvență: Căutare produs (Client)

5. Vizualizare catalog produse (Client)

Cererea este preluată de controller, iar serviciul de produse extrage integral catalogul din baza de date pentru a-l trimite în frontend.

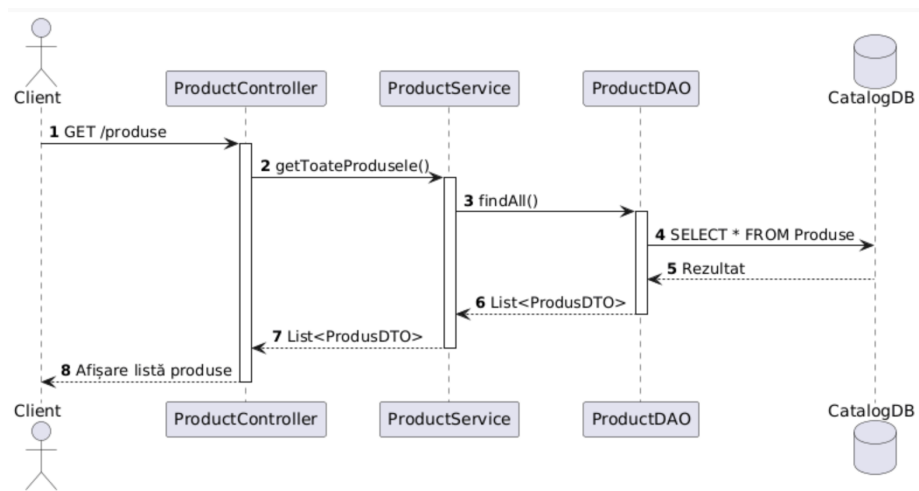


Figure 32: Diagrama de secvență: Vizualizare catalog public

6. Filtrare produse în magazin (Angajat)

Filtrele aplicate de angajat sunt trimise către serviciul de inventar, care interoghează baza de date folosind clauze specifice pentru magazinul respectiv.

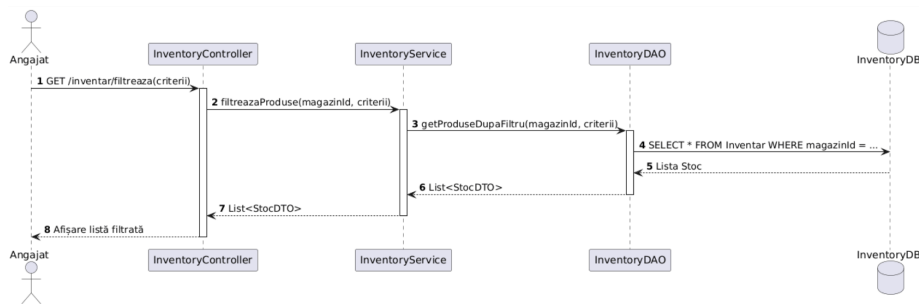


Figure 33: Diagrama de secvență: Filtrare produse magazin

7. Export utilizatori (Administrator)

Administratorul cere exportul. Sistemul extrage lista din baza de date a utilizatorilor, o convertește în șiruri CSV și trimite fișierul generat.

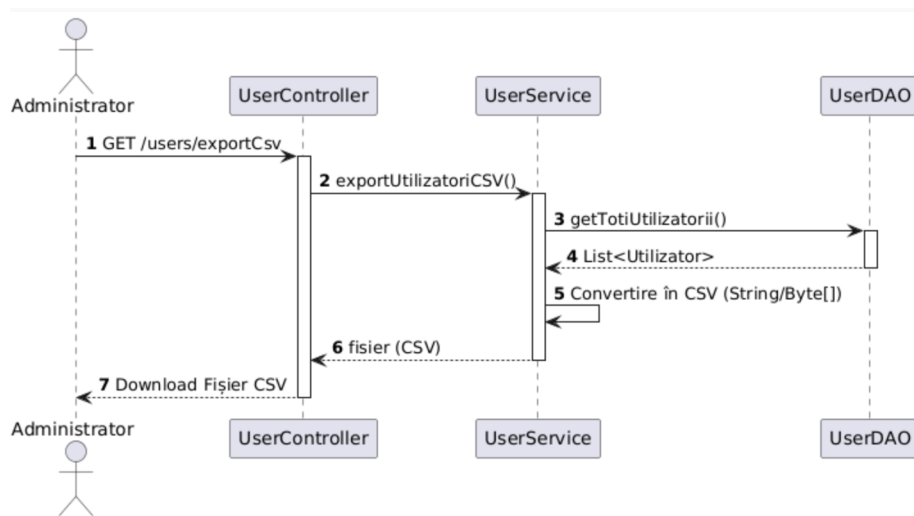


Figure 34: Diagrama de secvență: Export utilizatori (CSV)

8. Ștergere utilizator (Administrator)

Se trimite comanda de ștergere cu ID-ul contului, serviciul validează cererea și elimină înregistrarea din baza de date.

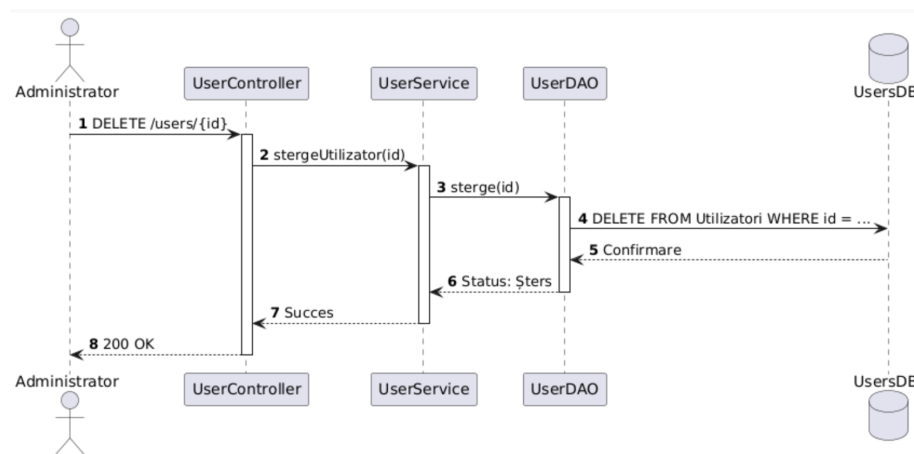


Figure 35: Diagrama de secvență: Ștergere cont utilizator

9. Autentificarea utilizatorului

Datele de conectare sunt verificate în tabelul de utilizatori; dacă informațiile sunt valide, se emite un răspuns de succes cu detaliile de sesiune.

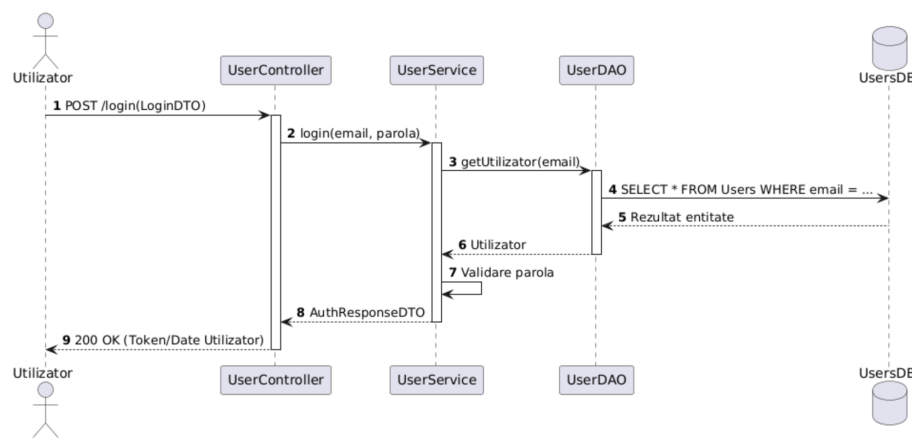


Figure 36: Diagrama de secvență: Autentificare

10. Vânzarea unui produs (Angajat)

La confirmarea vânzării, serviciul accesează inventarul și aplică o reducere a stocului proporțională cu cantitatea vândută.

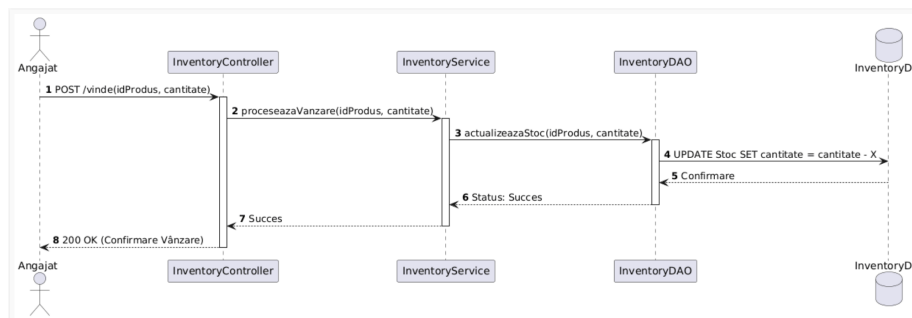


Figure 37: Diagrama de secvență: Vânzarea unui produs

11. Căutare avansată produs (Angajat)

Se analizează stocul local. Dacă rezultatul este zero, serviciul extinde căutarea în rețea printr-un apel intern către baza de date centrală de stocuri.

12. Filtrare produse (Manager)

Cereri globale de filtrare sunt direcționate către baza principală de produse pentru a afișa doar cosmeticele care corespund criteriilor selectate.

13. Operații CRUD produse (Manager)

Adăugarea unui produs implică primirea DTO-ului, transformarea acestuia în entitate, și salvarea permanentă în baza de date a catalogului.

14. Generare rapoarte (Manager)

Când este solicitat un anumit tip de fișier, serviciul folosește o clasă Factory pentru a instanția clasa de export corectă (XML, JSON etc.) și returnează datele.

15. Actualizare cont și notificare (Administrator)

Modificarea contului se face în UserService, după care se declanșează NotificationDispatcher pentru a trimite alertele către EmailObserver și SMSObserver.

5.2.3 Diagrama de componente

Diagrama de componente ne ajută să vizualizăm arhitectura la un nivel înalt, demonstrând divizarea în microservicii.

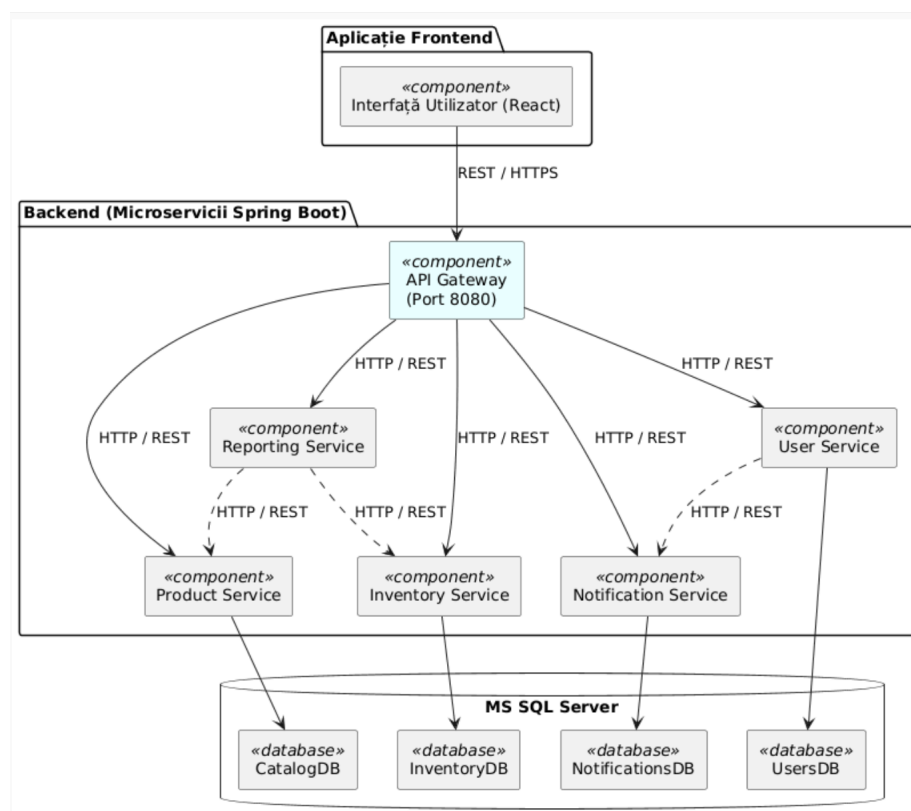


Figure 38: Diagrama de componente a sistemului

Observăm trei straturi majore:

- **Aplicația Frontend (React):** Gestionează interfața grafică și trimite toate cererile utilizatorilor către un singur punct central (API Gateway), folosind protocolul REST/HTTPS.

- **Backend-ul (Spring Boot):** Conține API Gateway-ul și microserviciile independente. Gateway-ul funcționează ca o ușă principală”, preluând cererile de la frontend și direcționându-le automat către microserviciul potrivit. Microserviciile rezolvă logica de afaceri și pot comunica între ele prin apeluri interne HTTP atunci când un flux o cere.
- **Baza de Date (MS SQL Server):** Demonstrează respectarea principiilor arhitecturii de microservicii, fiecare componentă având propria bază de date (cu excepția modulului de raportare, care doar procesează date existente).

5.2.4 Diagrama de dezvoltare (Deployment)

Această diagramă ilustrează infrastructura tehnică de găzduire.

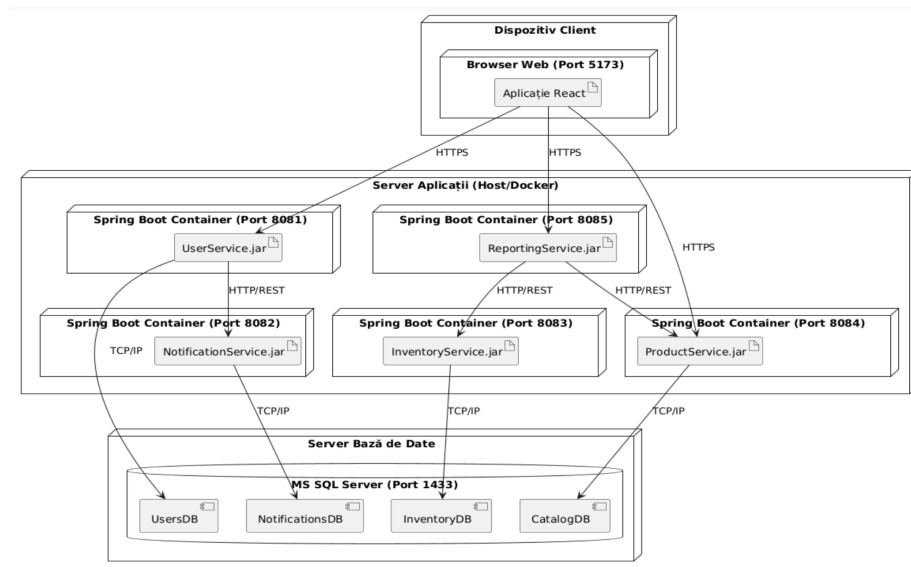


Figure 39: Diagrama de dezvoltare (Deployment Diagram)

Structura se bazează pe containere izolate (Docker) plasate pe un Server de Aplicații. Acest lucru asigură o rulare independentă, pe porturi dedicate, pentru fiecare modul în parte. Browserul web accesează în mod securizat (HTTPS) exclusiv containerul API Gateway, care ascultă pe portul 8080.

Mai departe, Gateway-ul direcționează traficul prin rețeaua internă a serverului către containerele microserviciilor (porturile 8081-8085). La rândul lor, toate microserviciile de backend accesează serverul central MS SQL prin rețeaua internă (portul 1433, protocol TCP/IP). Această abordare limitează expunerea directă a bazelor de date și a serviciilor interne către internet, obținând un nivel ridicat de performanță și securitate.

6 Descrierea aplicației

6.1 Structura aplicației Frontend

Codul sursă al aplicației client (dezvoltat în React) este organizat modular în interiorul directorului principal `src`. Această împărțire separă clar responsabilitățile, facilitând o dezvoltare ordonată și o mentenanță ușoară a proiectului.

Principalele module ale aplicației sunt structurate astfel:

- `api/`: Conține fișierul `backend.js`, care acționează ca un punct central pentru toate apelurile de rețea (HTTP). Acesta gestionează comunicarea cu serverul, trimițând cererile către API Gateway.
- `components/`: Grupează componentele vizuale independente și reutilizabile. Aici se regăsesc elemente precum bara de navigare (`Navbar.jsx`), formatul de afișare al unui produs (`ProductCard.jsx`) și componentele de bază ale interfeței (butoane, câmpuri de text, grupate în `ui.jsx`).
- `context/`: Găzduiește `AppContext.jsx`, responsabil pentru managementul stării globale a aplicației. Acesta păstrează în memorie datele esențiale pentru sesiunea curentă, cum ar fi detaliile utilizatorului conectat și limba selectată pentru interfață.
- `i18n/`: Conține fișierul `translations.js`, care asigură suportul multilingv al aplicației, stocând textele pentru interfață în diferite limbi (română, engleză, franceză).
- `pages/`: Reprezintă ecranele mari ale aplicației. Fiecare fișier de aici (de exemplu, `LoginPage.jsx`, `CatalogPage.jsx`, `AdminPage.jsx`, `StatsPage.jsx`) corespunde unei secțiuni distincte din platformă, asamblând componentele mai mici pentru a construi paginile specifice fiecărui tip de utilizator.

Această organizare a fișierelor permite o navigare rapidă în cod și asigură o delimitare corectă între logica de preluare a datelor, managementul stării și afișarea interfeței grafice.

Aplicația oferă o interfață grafică prietenoasă, structurată pe module specifice pentru fiecare tip de utilizator în parte.

6.2 Autentificarea

Punctul de intrare pentru personal. Utilizatorul furnizează datele, iar sistemul gestionează accesul bazat pe roluri (Angajat, Manager, Administrator).

6.3 Interfața Clientului

Pagina principală permite vizitatorilor să navigheze liber prin catalogul de produse cosmetice și să verifice disponibilitatea lor.

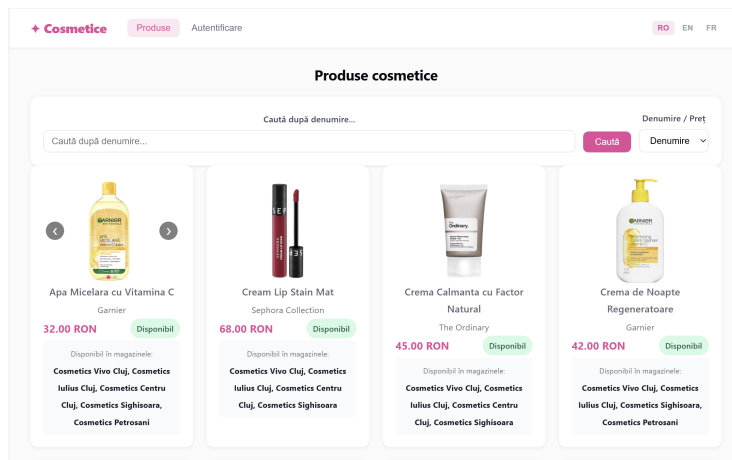


Figure 40: Catalogul public pentru clienți

6.4 Interfața Angajatului

Concepută pentru eficiență, oferă angajatului opțiuni rapide pentru a marca produse ca fiind vândute sau pentru a introduce stocuri noi.

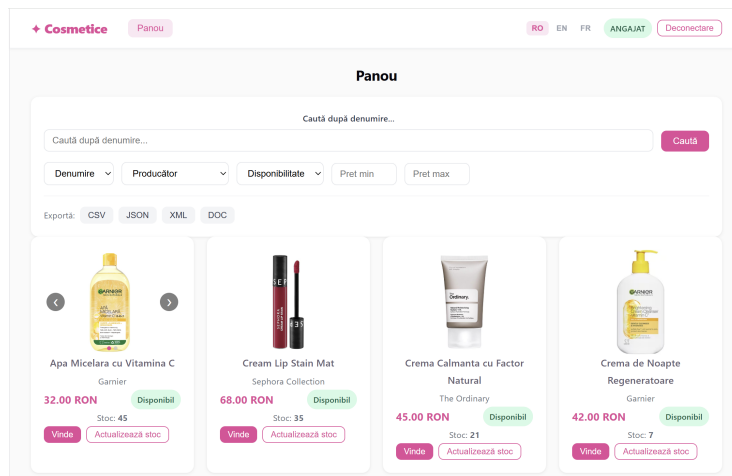


Figure 41: Panoul de inventar pentru Angajat

6.5 Interfața Managerului

Un panou administrativ centralizat de unde managerul adaugă produse noi în catalog sau editează specificațiile celor existente.

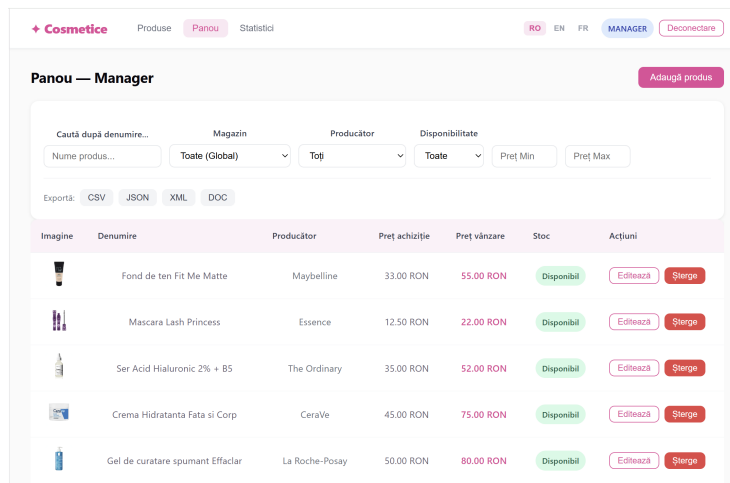


Figure 42: Panoul de administrare produse pentru Manager

6.6 Interfața Administratorului

Dedicată exclusiv managementului de personal, permite vizualizarea clară a conturilor angajaților, cu posibilități de filtrare și editare a drepturilor.

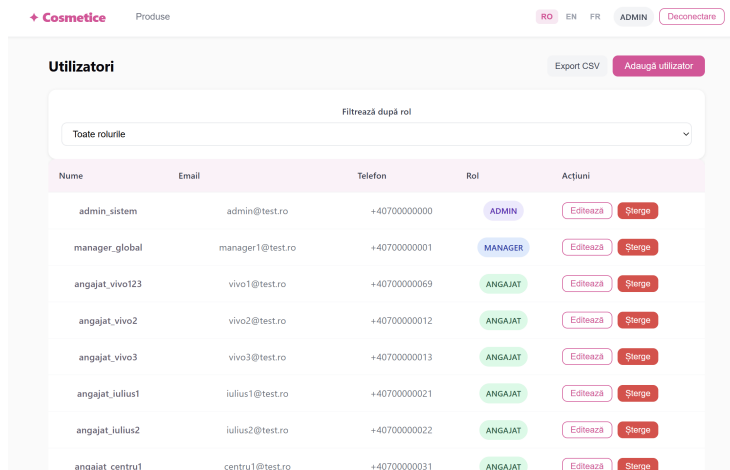


Figure 43: Panoul de gestionare conturi pentru Administrator

6.7 Grafice și Statistici

Managerul vizualizează rapoartele sub formă de grafice interactive. Acestea evidențiază trendurile vânzărilor, mărcile de succes și situația stocurilor.